Afar-PrgEnv: a configurable programming environment for AMD Afar compilers for Cray EX systems

Oscar Hernandez
oscar@ornl.gov
Oak Ridge National Laboratory
Oak Ridge, Tennessee, USA

Wael Elwasif
elwasif@ornl.gov
Oak Ridge National Laboratory
Oak Ridge, Tennessee, USA

Abstract

Afar-prgenv is a module-based overlay that enables early evaluation of AMD compiler drops on Cray/HPE systems without disrupting the Cray Programming Environment. The tool integrates AFAR compiler and ROCm drops into the PE hierarchy, preserves the ‘ftn/cc/CC‘ workflow, and stabilizes MPI and library metadata through pkg-config shims and wrapper diagnostics. We describe the design and implementation of Afar-prgenv and evaluate it on Frontier using the current PE and AFAR drop series. Our evaluation uses a reproducible test matrix covering MPI, OpenMP offload, and key Cray libraries (HDF5, FFTW, LibSci, DSMML, and PnetCDF), followed by extended validation with the SOLLVE suite and an OLCF Fortran application set. The results show that Afar-prgenv makes early compiler drops immediately usable with production libraries and support tools, providing a repeatable path to certify monthly drops before general availability. This work positions Afar-prgenv as a user support tool that bridges the gap between vendor release cadence and production readiness.

ACM Reference Format:
Oscar Hernandez and Wael Elwasif. 2026. Afar-PrgEnv: a configurable programming environment for AMD Afar compilers for Cray EX systems. In *Proceedings of 12th International Workshop on HPC User Support Tools (SC25) (HUST ’25)*. ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Cray/HPE systems provide a stable programming environment (PE) that couples compilers, MPI, and optimized libraries behind consistent ‘ftn/cc/CC‘ wrappers. This stability is essential for production workloads, but it slows adoption of new compiler drops from AMD, particularly when the Cray PE does not yet ship the corresponding toolchain. Support teams and early adopters are left with a choice: wait for vendor integration or accept a fragmented environment that cannot interoperate with PE libraries and tooling.

Afar-prgenv closes this gap.[1] It overlays AMD AFAR compiler and ROCm drops into the Cray PE module hierarchy while preserving the PE contract that users and build systems depend on. The overlay provides wrapper logic for OpenMP offload, MPI metadata re-synchronization, and pkg-config shims that keep library flags stable even when modules are swapped. The result is a workflow where new drops can be tested monthly without breaking production tooling, and without requiring users to abandon the PE conventions they rely on.

1.1 Contributions

- Define the user-support problem for early compiler evaluation on Cray/HPE systems and motivate the need for a safe, reversible overlay.
- Present the Afar-prgenv design and implementation, including its module hierarchy, wrapper behavior, and pkg-config integration.
- Evaluate Afar-prgenv on Frontier with representative compile, link, and offload workflows, and report operational outcomes for support teams.

2 Motivation

Frontier and similar Cray/HPE systems evolve rapidly, while vendor-supported programming environments lag behind emerging AMD compiler and ROCm drops. User support teams need a mechanism to validate new compiler stacks without disrupting production workflows or forcing users to rebuild their toolchains from scratch.

AFAR drops arrive on a roughly monthly cadence. Each drop must be validated for basic functionality, compatibility with the PE module hierarchy, and correctness in mixed environments where MPI and vendor libraries are required. Without an overlay, AFAR drops are effectively isolated: they cannot cleanly consume Cray libraries or interact with standard build systems that rely on `ftn/cc/CC`. In practice, this makes AFAR drops unusable for production users until a support tool like Afar-prgenv bridges the gap.

2.1 User Support Challenges

Support teams must bridge mismatched compiler/runtime versions, conflicting modules, and inconsistent compile/link metadata across rapidly changing software stacks. Even when an AFAR drop compiles user code, it can fail at link time due to missing PE library metadata or incompatible Fortran modules. These issues are difficult to triage because the failure surface spans compiler wrappers, MPI module files, and library search paths.

2.2 Use Cases

- Early validation of AMD compiler drops against Cray PE workflows.
- Compatibility testing for MPI, OpenMP offload, and vendor libraries.

Unpublished working draft. Not for distribution.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
HUST ’25, St. Louis, MO, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Unpublished working draft. Not for distribution.

- Reproducible support environments for debugging user issues.

3 Background

This section summarizes the Cray Programming Environment, the AFAR compiler drop model, and the Frontier system context used in this study.

3.1 Cray Programming Environment

The Cray PE exposes consistent compiler drivers (`ftn`, `cc`, and `CC`) backed by a module hierarchy. Users load a `cpe/<ver>` module and a corresponding `PrgEnv-*` module (e.g., `PrgEnv-amd/8.6.0`) which configure compiler wrappers, MPI integration, and library metadata. The wrappers embed PE-specific assumptions about flag handling and `PKG_CONFIG_PATH` (including the `PE_AMD_FIXED_PKGCONFIG_PATH` convention), and most build systems assume those semantics remain stable.

3.2 AMD Compiler and ROCm Drops

AFAR drops bundle AMD compilers with a matched ROCm toolkit, and they are released on a rapid cadence. Each drop requires validation against Cray PE libraries and MPI, but the drops themselves do not integrate with the PE module hierarchy. Afar-prgenv provides that integration layer while preserving PE wrapper behavior.

At the time of this evaluation, the latest AFAR drop available on Frontier is 22.3.0-10004, paired with `rocm/7.0.2` and `amd/7.0.2`. While AFAR drops ship their own ROCm runtime, site workflows may still load a system ROCm module (such as 'rocm/7.0.2') to satisfy dependencies of selected vendor libraries that lag behind the drop.

3.3 Frontier as a Target System

Frontier is an HPE Cray EX system with AMD GPUs. The system uses a module-driven PE; at the time of this work, `cpe/25.09` and `PrgEnv-amd/8.6.0` are available. The recommended baseline for AFAR testing in this workspace loads `rocm/7.0.2` and `amd/7.0.2` alongside `PrgEnv-amd`, then adds the AFAR module tree from `/sw/crusher/ums/compilers/modulefiles/`.

4 Related Work

Module systems such as Lmod underpin most HPC software stacks, and package managers like Spack and EasyBuild automate dependency resolution and software installation. These tools are essential for building and distributing software, but they do not address PE-specific wrapper semantics or MPI metadata coupling that Cray/HPE systems rely on.

Compiler wrapper techniques are widely used to inject site defaults or architecture flags, yet they typically assume a single compiler stack rather than a layered environment that overlays experimental toolchains onto a production PE. Afar-prgenv complements these approaches by focusing on the user-support problem of early compiler evaluation in a production PE, with an emphasis on preserving the 'ftn/cc/CC' contract and vendor library integration.

5 Design Goals

The design targets minimal disruption to the Cray PE workflow, reproducibility for support teams, and portability across Cray/HPE systems. The primary assumption is that user build systems and support tooling already depend on PE wrappers and module semantics.

- Preserve Cray wrapper semantics while enabling AMD compiler drops.
- Keep module interactions deterministic and reversible.
- Provide consistent MPI and library metadata via pkg-config.
- Support rapid testing with clear diagnostics for support staff.

5.1 Non-Goals

We do not replace the Cray PE or provide a new package manager. Instead, we focus on the narrow but critical gap between AFAR drop availability and PE integration, prioritizing correctness and compatibility over performance tuning.

6 Architecture and Implementation

Afar-prgenv integrates AMD compiler drops into the Cray PE by providing a module hierarchy and lightweight wrapper logic. The overlay is intentionally minimal: it avoids replacing the PE and instead augments it with AFAR-specific paths, metadata, and diagnostics.

6.1 Module Hierarchy

The module tree is split into a meta module (`afar-prgenv/<ver>`) and two component modules: `afar-rocm/<ver>` and `afar-amd/<ver>`. The meta module unloads conflicting vendor modules, then loads the ROCm toolkit module followed by the compiler module to ensure the compilers are last in 'PATH'. This keeps the Cray PE wrappers stable while redirecting the underlying toolchain.

6.2 Compiler Wrappers

Afar-prgenv ships 'ftn', 'cc', and 'CC' wrappers that preserve Cray PE behavior while injecting AFAR-specific fixes. The wrappers:

- Resolve the real Cray compiler driver and delegate to it.
- Clear `CRAY_ACCEL_*` to avoid unsupported PE offload flags.
- Inject '–offload-arch' when '-fopenmp' is present and no offload arch is explicitly specified.
- Re-synchronize MPI metadata at invocation time to match the active 'cray-mpich' module.
- Filter or replace incompatible flags using a configurable rule set.

6.3 pkg-config Integration

MPI and vendor libraries depend on consistent 'pkg-config' metadata. The generator produces AFAR-specific '.pc' files for MPI and ROCm, while a shim generator captures module-provided metadata for Cray libraries that do not ship usable '.pc' files. The wrappers mirror the PE-specific metadata path into `PKG_CONFIG_PATH` to keep build systems stable under module swaps. When required ROCm 6 runtime libraries are missing, the AFAR ROCm module suppresses Cray MPICH GTL 'pkg-config' variables to avoid injecting incompatible link flags. This keeps MPI builds functional

Unpublished working draft. Not for distribution.

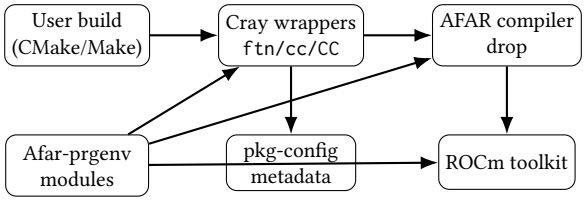

Figure 1: Afar-prgenv overlay architecture. The Cray PE remains the primary interface; Afar-prgenv inserts AFAR compilers and ROCm while preserving PE wrapper semantics and library metadata.

while signaling that accelerator-enabled MPI paths need follow-up validation.

6.4 Generation and Configuration

Version mappings and PE paths are defined in configuration files and are regenerated with a single script. This keeps the module tree reproducible and aligns the AFAR drop version with the ROCm toolkit and MPI metadata used in tests. The generator also writes diagnostics that the support team uses to track drop readiness over time.

7 Operational Workflow

This section describes how support teams deploy Afar-prgenv and how users interact with it during early drop evaluation.

7.1 Support Team Workflow

Support staff add a new AFAR drop under the configured root, update the version mapping, and regenerate the module tree using `afar_modules/scripts/generate_afar_modules.sh`. They then run the repro test suite via `scripts/run_afar_tests.sh -keep-going` and capture logs for validation. A drop is considered ready for user evaluation when the repro matrix passes and the wrapper diagnostics show stable MPI and library metadata. The harness writes per-profile logs under `logs/`, which serve as support artifacts for regression tracking.

After repro gating, support staff run extended validation suites, including an OLCF Fortran application set and the SOLLVE validation suite, to assess production readiness beyond basic compile and link coverage.

7.2 User Workflow

Users follow the standard PE module sequence (e.g., 'cpe/25.09', 'PrgEnv-amd/8.6.0') and then load 'afar-prgenv/22.3.0-10004'. The wrappers preserve the expected compile and link commands while injecting AFAR paths and offload flags when needed.

7.3 Failure Modes and Diagnostics

Common failure modes include missing or mismatched MPI metadata, incompatible Fortran module files from vendor libraries, and absent ROCm runtime libraries. Wrapper diagnostics surface these issues early by reporting filtered flags, active MPI flavors, and updated `PKG_CONFIG_PATH` settings.

The repro harness also exercises different module load orders to ensure that the wrappers re-synchronize MPI and library metadata correctly when users load modules before or after 'afar-prgenv'.

8 Experimental Setup

We evaluate Afar-prgenv on Frontier using the current Cray PE and the latest AFAR drop available in the system module tree. The goal is to measure compatibility with the PE toolchain and key libraries, not peak performance.

8.1 Systems and Software

Our baseline environment loads 'cpe/25.09' and 'PrgEnv-amd/8.6.0' alongside 'rocm/7.0.2' and 'amd/7.0.2'. Afar-prgenv modules are provided from '/sw/crusher/ums/compilers/modulefiles/', and the evaluation targets 'afar-prgenv/22.3.0-10004'. This setup mirrors the site-recommended module sequence used by the support team.

Although the AFAR drop ships a ROCm runtime, the additional 'rocm/7.0.2' module is retained for compatibility with PE libraries that still depend on system ROCm paths. When evaluating older dependencies, we also verify compatibility against ROCm 6.x where needed. The repro harness focuses on PE and AFAR modules; ROCm compatibility for legacy dependencies is tracked as a separate gate (e.g., 'libsci-acc').

```
module purge
module reset
module load cpe/\peversion
module load rocm/\afarrocm
module load amd/\afarrocm
module load PrgEnv-amd
module load cray-mpich/9.0.1
module load craype-x86-trento
module load craype-accel-amd-gfx90a
module use /sw/crusher/ums/compilers/modulefiles/
module load afar-prgenv/\afarversion
```

8.2 Workloads and Repro Cases

We use the repository repro suite to validate compile and link correctness: 'compile-only', 'cmake-mpi-omp', 'wrapper-compile-link', 'mpi-hello', 'omp-c', 'omp-f90', 'libsci', 'libsci-acc', 'hdf5', 'fftw', 'dsmml', and 'netcdf'. The tests exercise the PE wrappers, MPI module files, and library metadata via `pkg-config`, and `scripts/check_ldd.sh` validates the resulting binaries. The CMake repro requires `cmake` in `PATH`.

The repro harness (`scripts/run_afar_tests.sh`) runs profiles that load each library module both before and after 'afar-prgenv', ensuring the wrapper re-synchronization logic works under module-order permutations.

In this study we ran:

```
scripts/run_afar_tests.sh --keep-going \
```

Unpublished working draft. Not for distribution.

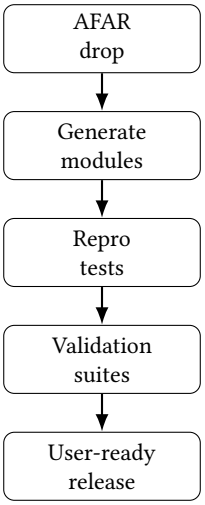

Figure 2: Monthly AFAR drop workflow. Afar-prgenv provides a repeatable path from drop ingestion to user readiness.

Table 1: Repro suite coverage for the AFAR drop 22.3.0-10004. Status is recorded as P/S/F for each test.

Test	Coverage	Status (P/S/F)
cmake-mpi-omp	CMake MPI+OMP	PASS
compile-only	compiler sanity	PASS
wrapper-compile-link	wrapper compile/link	PASS
mpi-hello	MPI module integration	PASS
omp-c	OpenMP offload (C)	PASS
omp-f90	OpenMP offload (F)	PASS
libsci	PE math libs	PASS
libsci-acc	accelerator libs	SKIP
hdf5-parallel	HDF5 (par)	SKIP
hdf5-serial	HDF5 (ser)	PASS
fftw	FFTW (C/C++/F)	PASS
dsmml	DSMML	PASS
netcdf	PnetCDF	PASS

```
--afar-version \afarversion \
--afar-module afar-prgenv/\afarversion
```

8.3 Metrics

The primary metric is functional readiness: all repro cases must compile and link with the PE wrappers and AFAR drop. Secondary metrics include the diagnostic completeness of wrapper logs, P/S/F rates per test, and the time to validate a new drop. We also track module-order robustness by running the repro suite under multiple module load permutations to ensure metadata re-synchronization remains correct.

9 Evaluation and Results

We evaluate Afar-prgenv against the functional readiness criteria defined in Section 8.3. The emphasis is on correctness and compatibility with PE libraries and MPI, which are prerequisites for any further performance or application-level validation.

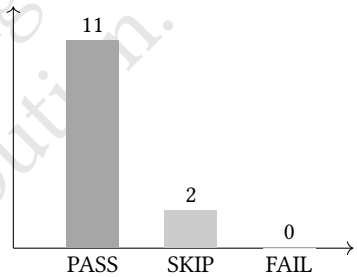

Figure 3: Repro suite outcomes for 22.3.0-10004(unique tests).

The 'libsci-acc' profile is intentionally skipped; `libsci_acc` linking is not yet validated and may depend on ROCm 6 runtime libraries. The parallel HDF5 test is skipped due to missing HDF5 C++ 'pkg-config' metadata, while the serial HDF5 path passes.

9.1 Compilation and Linking

The repro matrix validates that AFAR drops compile and link with the PE wrappers without requiring manual flag surgery. Wrapper diagnostics report any filtered flags or mismatched MPI metadata early, which reduces support turnaround when a drop regresses. All workflow-focused repros ('compile-only', 'cmake-mpi-omp', and 'wrapper-compile-link') pass for 22.3.0-10004. The CMake repro confirms that build systems can consume the wrappers without manual flag translation.

9.2 Module Ordering Robustness

To reflect real user behavior, we execute the repro suite across multiple module load permutations (e.g., library modules loaded before or after 'afar-prgenv'). This checks that wrapper logic correctly re-synchronizes MPI and library metadata regardless of module ordering. We report P/S/F outcomes per permutation and flag any regressions for follow-up.

Unpublished working draft. Not for distribution.

Table 2: Module ordering permutations used to validate metadata re-synchronization.

Permutation	Example	Status (P/S/F)
Libs before AFAR	load libs, then afar-prgenv	PASS
Libs after AFAR	load afar-prgenv, then libs	PASS
MPI swap after AFAR	swap cray-mpich post-load	N/A

The before/after permutations pass for the library profiles exercised in this study (LibSci, FFTW, DSMML, PnetCDF, and serial HDF5). MPI swap after AFAR is a planned check and was not exercised in this run.

9.3 MPI and Offload Validation

MPI validation confirms that the wrappers resynchronize the MPI flavor on each invocation, preventing stale 'pkg-config' metadata when modules are swapped. OpenMP offload tests confirm that '–offload-arch' is injected for both C and Fortran workflows. This is exercised with the PE module `craype-accel-amd-gfx90a`. Overrides via `AFAR__OFFLOAD_ARCH` are also covered. The 'mpi-hello', 'omp-c', and 'omp-f90' repros all pass in this evaluation.

9.4 Library Compatibility

Library repro tests confirm that AFAR drops can link against PE-provided HDF5, FFTW, LibSci, DSMML, and PnetCDF libraries when the correct 'pkg-config' metadata is present. The 'libsci-acc' path is treated as a conditional gate because ROCm runtime dependencies can lag behind AFAR drops. When the expected ROCm runtime libraries are present, 'libsci-acc' builds successfully; otherwise the test is skipped and the drop is held for follow-up. The 'libsci-acc' repro uses `AFAR_LIBSCI_ACC_LIBNAME` to target the correct accelerator library variant.

HDF5 validation uses an AFAR-compiled Fortran shim module to avoid incompatible vendor '.mod' files while still linking against the HDF5 C libraries. The PnetCDF repro relies on 'pnetcdf.inc' instead of compiler-specific '.mod' files, and it skips Fortran if the include file is absent. FFTW and DSMML validation use 'pkg-config' metadata and `scripts/check_ldd.sh` to ensure the link paths resolve to the expected PE libraries.

9.5 Operational Impact

Operationally, Afar-prgenv converts AFAR drops from isolated toolchains into PE-compatible environments. Support teams can validate a drop with a deterministic repro suite and only escalate to broader validation (Fortran test suites, OpenMP validation, and application builds) once the PE compatibility baseline is satisfied. In this run, the repro suite reported no failures; the only skips were intentional gates for 'libsci-acc' and the missing HDF5 C++ 'pkg-config' data in the parallel HDF5 path.

9.6 Extended Validation

After repro gating, we run an OLCF Fortran application set and the SOLLVE validation suite to evaluate readiness for production use. These suites provide broader coverage of Fortran runtime behavior, OpenMP offload correctness, and application build workflows that are not captured by the repro tests alone.

Table 3: Extended validation suites used after repro gating.

Suite	Coverage	Status (P/S/F)
OLCF Fortran apps	Fortran build/runtime	TBD
SOLLVE V&V	OpenMP offload	TBD

10 Discussion

We discuss the trade-offs of the overlay approach, portability to other Cray/HPE systems, and limitations observed during validation.

10.1 Limitations

The overlay depends on PE wrappers and module hierarchy conventions; changes in those interfaces require updates to Afar-prgenv. Some vendor libraries ship Fortran modules that are not compatible with AFAR compilers and require shims or local builds. Accelerator libraries (e.g., 'libsci-acc') may also lag when runtime dependencies are tied to older ROCm versions. Parallel HDF5 validation can also be constrained by missing C++ 'pkg-config' metadata in the PE environment, which must be addressed before full coverage.

We also intentionally avoid performance benchmarking in this work. Our goal is functional readiness and compatibility; performance evaluation is deferred until after a drop passes the compatibility gate.

10.2 Portability

Afar-prgenv generalizes to other Cray/HPE systems because it leverages the same PE module hierarchy and wrappers. The system-specific configuration is limited to the AFAR root path, version mappings, and the Cray MPICH install path, all of which are captured in configuration files.

10.3 Lessons Learned

Two operational lessons stand out. First, early drop readiness is mostly a metadata problem: wrapper semantics and 'pkg-config' paths must be correct before any application testing can succeed. Second, reproducible repro tests are critical for support teams because they isolate regressions to specific libraries or wrapper behaviors. These lessons inform how we prioritize future automation and validation.

11 Conclusion and Future Work

Afar-prgenv enables early evaluation of AMD compiler drops on Cray/HPE systems by preserving the PE workflow while integrating AFAR compilers and ROCm toolkits. The overlay converts isolated drops into PE-compatible environments that can be validated with reproducible repro tests, giving support teams a repeatable path to certify monthly drops before broader application testing.

Future work includes expanding the validation suite to cover additional Fortran and OpenMP test suites, integrating application-level regression tests, and automating readiness reports for new drops.

Acknowledgments

Please add

Unpublished working draft. Not for distribution.

References

[1] Oscar Hernandez and Wael Elwasif. 2025. Afar-prgenv: A configurable programming environment for AMD AFAR drops on Cray HPE systems. Internal tool used at OLCF Frontier.

Unpublished working draft.
Not for distribution.